

Class 7 Machine Learning 1

Qiman A17912409

Table of contents

Background	1
K-means clustering	1
Hierarchical Clustering	7
Analysis of UK food data	9
Tidy the data	10
Exploratory Analysis	11

Background

Today we will explore some core machine learning methods that are very popular in bioinformatics. These include **clustering** and **dimensionality reduction**.

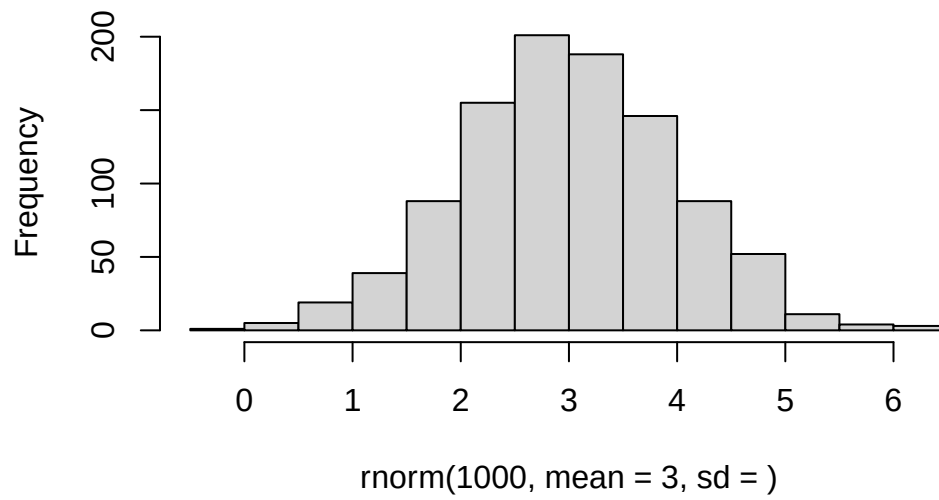
K-means clustering

The main function in “base” R for K-means clustering is called `kmeans()`

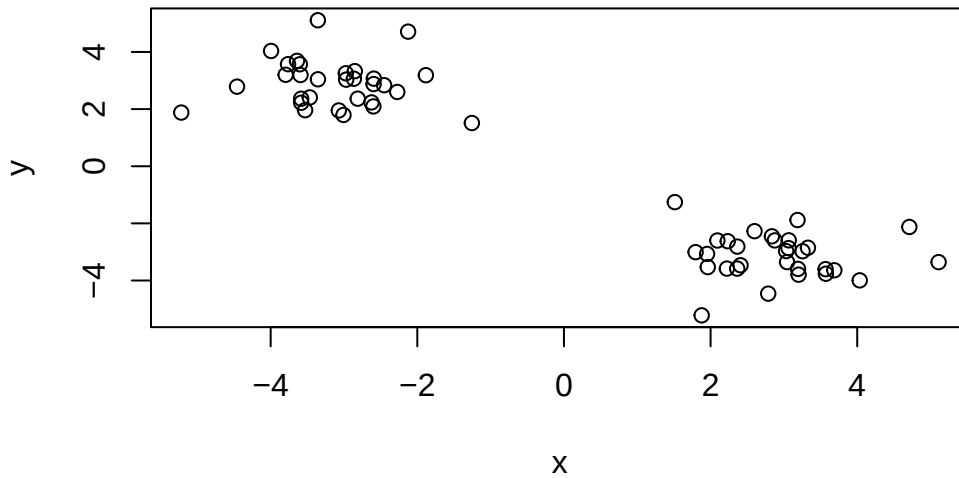
Before we go too deep let’s make up some “simple” data that we can cluster. To do this we can use the `rnorm()` function:

```
hist(rnorm(1000,mean=3, sd=))
```

Histogram of rnorm(1000, mean = 3, sd =)



```
x <- c(rnorm(30,-3),rnorm(30,+3))  
z <- cbind(x=x,y=rev(x))  
#rev(x)  
plot(z)
```



Now we can run `kmeans()` on this input `z` and see what the results look like.

```
km <- kmeans(z, centers=2)
km
```

K-means clustering with 2 clusters of sizes 30, 30

Cluster means:

	x	y
1	-3.130053	2.897808
2	2.897808	-3.130053

Clustering vector:

[illegible]

Within cluster sum of squares by cluster:

```
[1] 37.97986 37.97986
(between_SS / total_SS = 93.5 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
```

```
attributes(km)
```

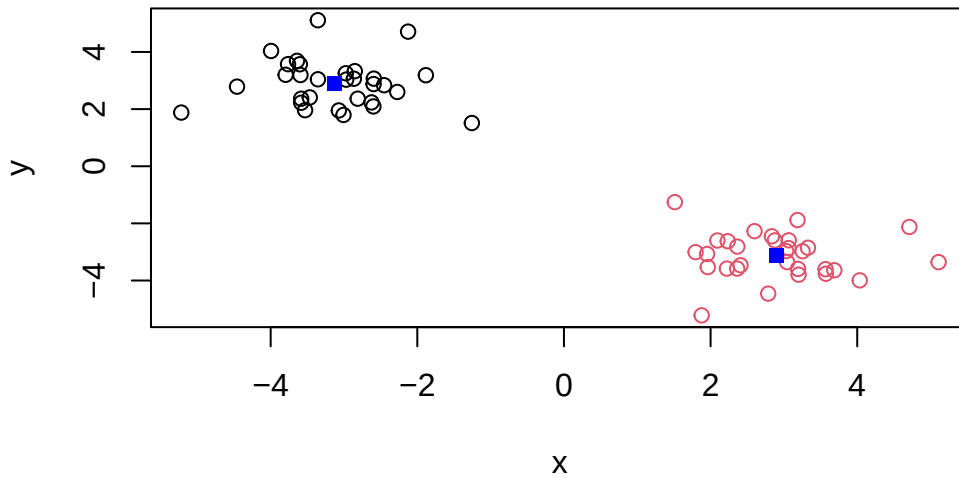
Q How many points are in each cluster?

Q. what component of your result object details cluster center?

Q. what component of your result object details cluster assignment/membership?

Q. PLOT `z` colored by the kmeans cluster assignment and add cluster centers as blue points

4



Q. Run a K-means clustering and plot the results asking for 4 clusters

```
km4 <- kmeans(z, centers=4)
km4
```

K-means clustering with 4 clusters of sizes 11, 11, 30, 8

Cluster means:

	x	y
1	2.623763	-2.447878
2	3.700407	-3.368658
3	-3.130053	2.897808
4	2.171044	-3.739963

Clustering vector:

[1]	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
[39]	2	1	4	1	2	4	1	1	2	1	4	2	1	1	2	1	2	2	4	2	2	1						

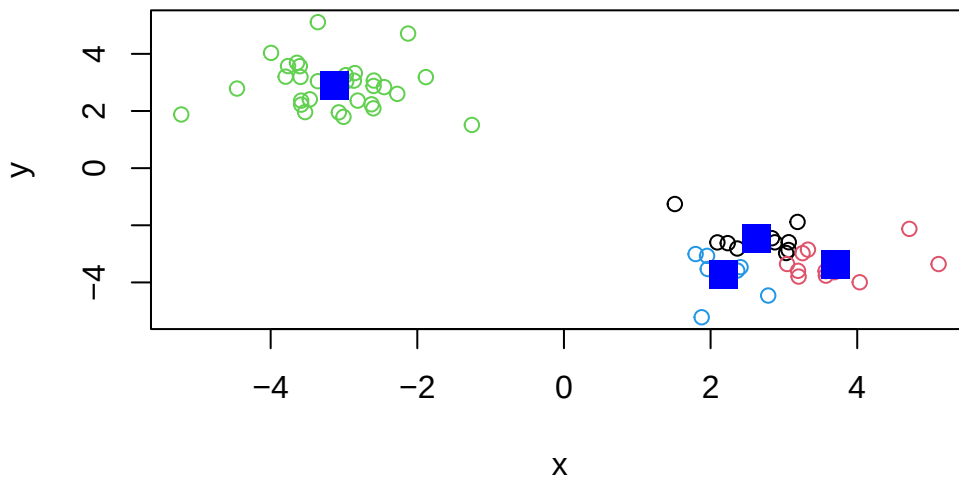
Within cluster sum of squares by cluster:

```
[1] 5.163676 7.306606 37.979860 4.650992
(between_SS / total_SS = 95.3 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

```
plot(z, col=km4$cluster)
points(km4$centers, col="blue", pch=15, cex=2)
```



N.B You need to tell K-means the number of clusters!!

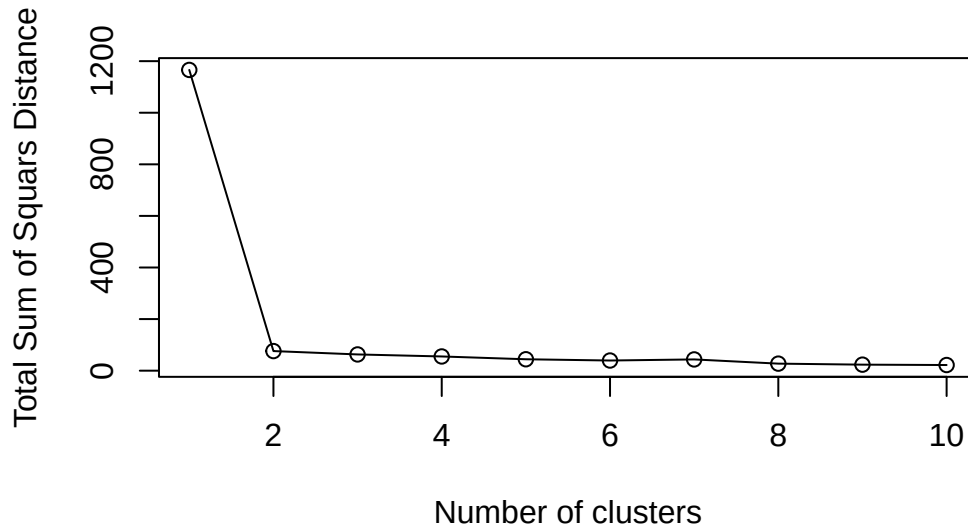
One approach is to try different values for `centers` and then pick the best

```
ans <- NULL
for(i in 1:10){
  km <- kmeans(z, centers=i)
  ans <- c(ans, km$tot.withinss)
}
```

```
ans
```

```
[1] 1166.01290  75.95972  62.95887  55.09666  44.24602  39.34736
[7]  43.65510  27.25648  23.40490  21.87197
```

```
plot(ans,typ="o", xlab="Number of clusters", ylab="Total Sum of Squars Distance")
```



Hierarchical Clustering

The main function in “base” R for Hierarchical Clustering is called `hclust()`

This function does not take your raw data for clustering. You must first build a “distance matrix” from your data and pass this as input to `hclust()`

```
d <- dist(z)
hc <- hclust(d)
hc
```

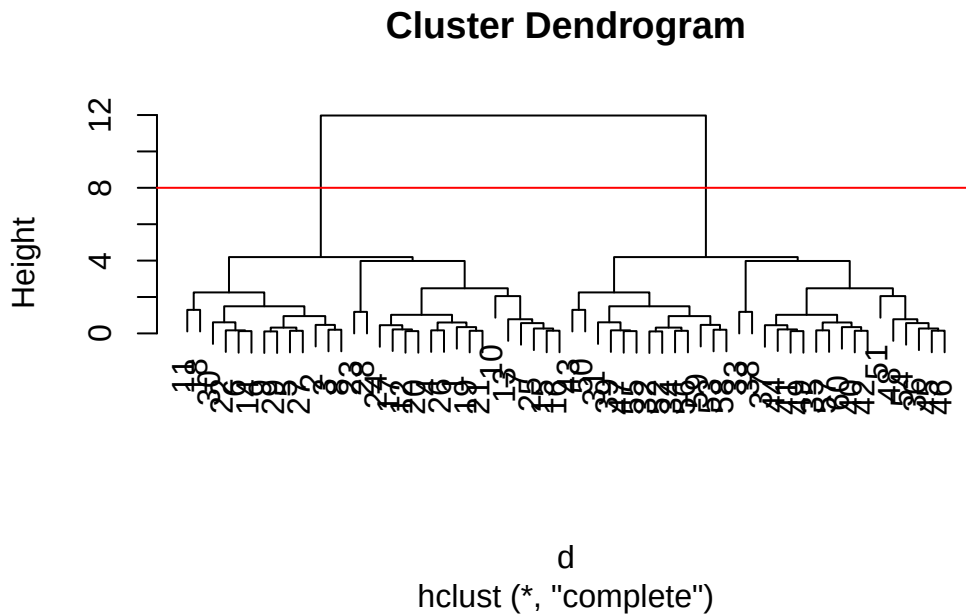
Call:

```
hclust(d = d)
```

```
Cluster method   : complete
Distance         : euclidean
Number of objects: 60
```

There is a `bespokeplot()` method for `hclust()` result objects.

```
plot(hc)
abline(h=8,col="red")
```



Once we have our `hclust` object (our tree of cluster dendrogram) we can **cut** the tree to reveal the clustering pattern.

```
cutree(hc, h=8)
```

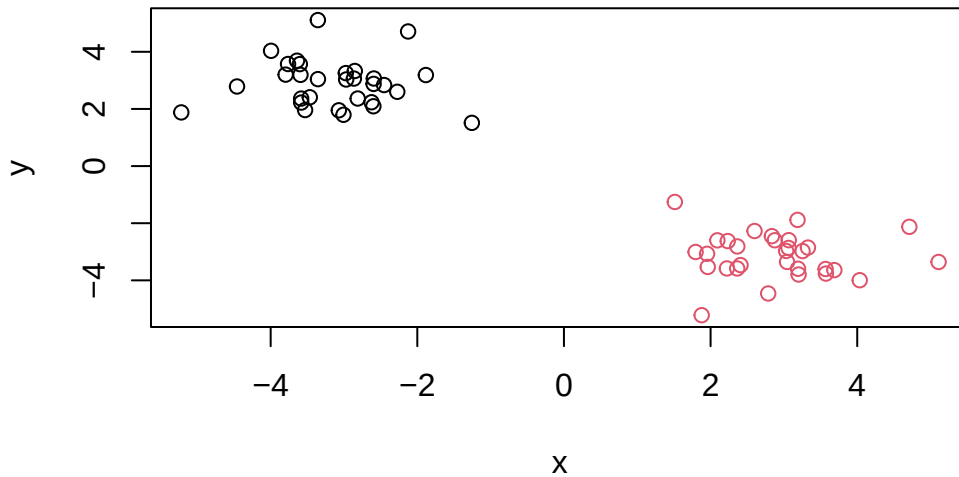
```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

```
cutree(hc, k=4)
```

```
[1] 1 2 2 1 2 2 1 2 2 1 2 1 1 2 1 1 1 2 1 1 1 1 2 1 2 2 3 3 4 3 4 4 4 4
[39] 3 4 4 4 3 4 4 4 3 4 4 3 3 4 3 3 4 3 3 4 3 3 4
```

Q make a plot of `z` with your `hclust` results(i.e. colored by cluster membership)

```
height <- cutree(hc, k=2)
plot(z, col=height)
```



##Principal Component Analysis(PCA)

PCA is a dimensionality reduction method that is popular for revealing patterns in complex datasets.

Analysis of UK food data

Let's look at some data on the eating habits of folks from the UK to see if there are patterns and trends that have some regions being distinct from others.

##Data Import

The data is made available in CSV format so we can use the `read.csv()` function for import to R:

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
```

Q1. How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

```
dim(x)
```

```
[1] 17  5
```

```
nrow(x)
```

```
[1] 17
```

```
ncol(x)
```

```
[1] 5
```

There are 17 rows and 5 columns.

Tidy the data

Fix anything that went wrong with the data import

```
head(x)
```

	X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66
2	Carcass_meat	245	227	242	267
3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139

There seems to be 5 columns instead of 4, we will fix that below:

```
rownames(x) <- x[,1]  
x <- x[,-1]  
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

Now let's check the dimensions again to make sure

```
dim(x)
```

```
[1] 17  4
```

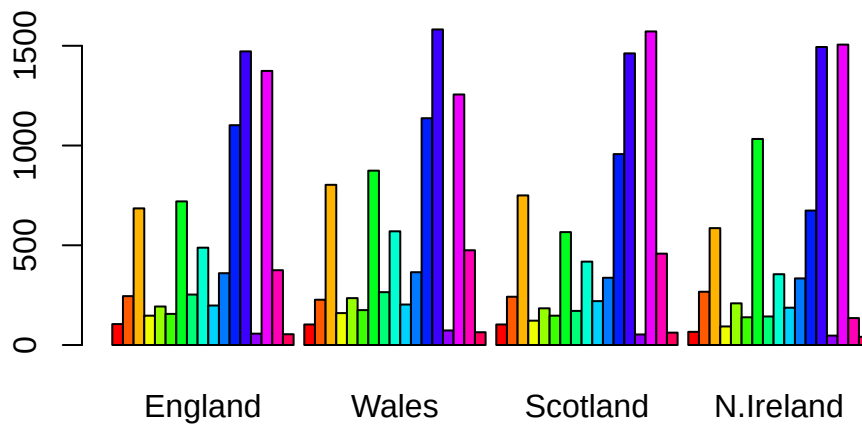
Q2. Which approach to solving the 'row-names problem' mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

I prefer the second approach where we read the dat file and set the row.names to be the first column. This approach seems better since it avoids problems with running the first approach code block multiple times. We just set the number of rows as 4 from the start.

Exploratory Analysis

Make some plots to help make sense of obvious trends. We will start with a comparative bar plot.

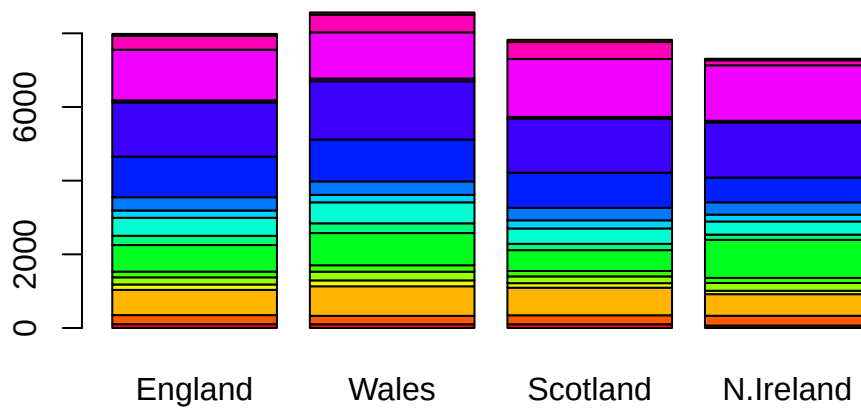
```
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```



Q3: Changing what optional argument in the above `barplot()` function results in the following plot?

Changing `beside=T` to `beside=F` in the `barplot()` code results in the following plot.

```
barplot(as.matrix(x), beside=F, col=rainbow(nrow(x)))
```

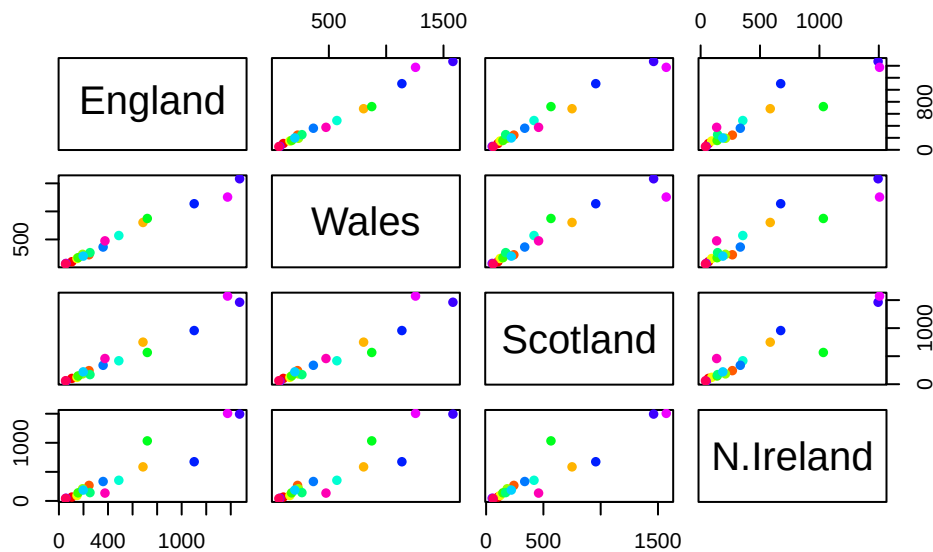


Q4: Changing what optional argument in the above `ggplot()` code results in a stacked barplot figure?

Changing the `geom_col(position="dodge")` to `position="stack"`

These plots don't really help, we can try a pairs plot.

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```

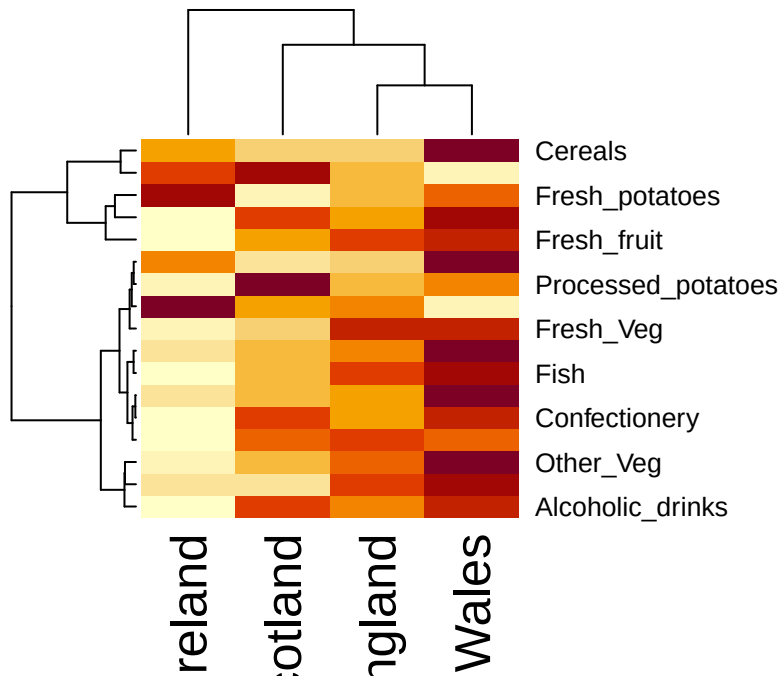


Q5: We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

The `pairs` function generates a matrix of scatterplots comparing all pairs of variables in the dataset. Each panel shows the relationship between two countries. A point on the diagonal indicates that the value for that observation is the same in both variables. This indicates identical consumption for that item.

Another useful visualization approach is a **heatmap**

```
heatmap( as.matrix(x) )
```



Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

According to the figures, England and Wales cluster together, this suggests that their consumption patterns are more similar. I can't easily tell what the main differences between N.Ireland and the other countries are.

##PCA to the rescue The main function in "base" R for PCA is called `prcomp()`. This function wants the "observations" to be in rows and the "variables" to be columns.

So here we need to take the transpose of our x input object:

```
t(x)
```

	Cheese	Carcass_meat	Other_meat	Fish	Fats_and_oils	Sugars
England	105	245	685	147	193	156
Wales	103	227	803	160	235	175
Scotland	103	242	750	122	184	147
N.Ireland	66	267	586	93	209	139
	Fresh_potatoes	Fresh_Veg	Other_Veg	Processed_potatoes		
England	720	253	488		198	
Wales	874	265	570		203	

Scotland	566	171	418		220
N.Ireland	1033	143	355		187
	Processed_Veg	Fresh_fruit	Cereals	Beverages	Soft_drinks
England	360	1102	1472	57	1374
Wales	365	1137	1582	73	1256
Scotland	337	957	1462	53	1572
N.Ireland	334	674	1494	47	1506
	Alcoholic_drinks	Confectionery			
England	375	54			
Wales	475	64			
Scotland	458	62			
N.Ireland	135	41			

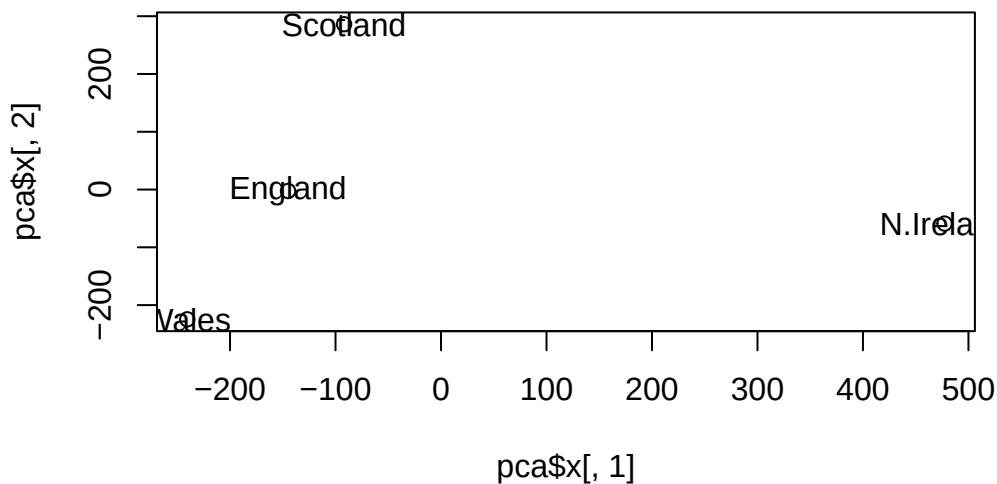
```
pca <- prcomp(t(x))
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	3.176e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

To make our new PCA plot we access `pca$x`

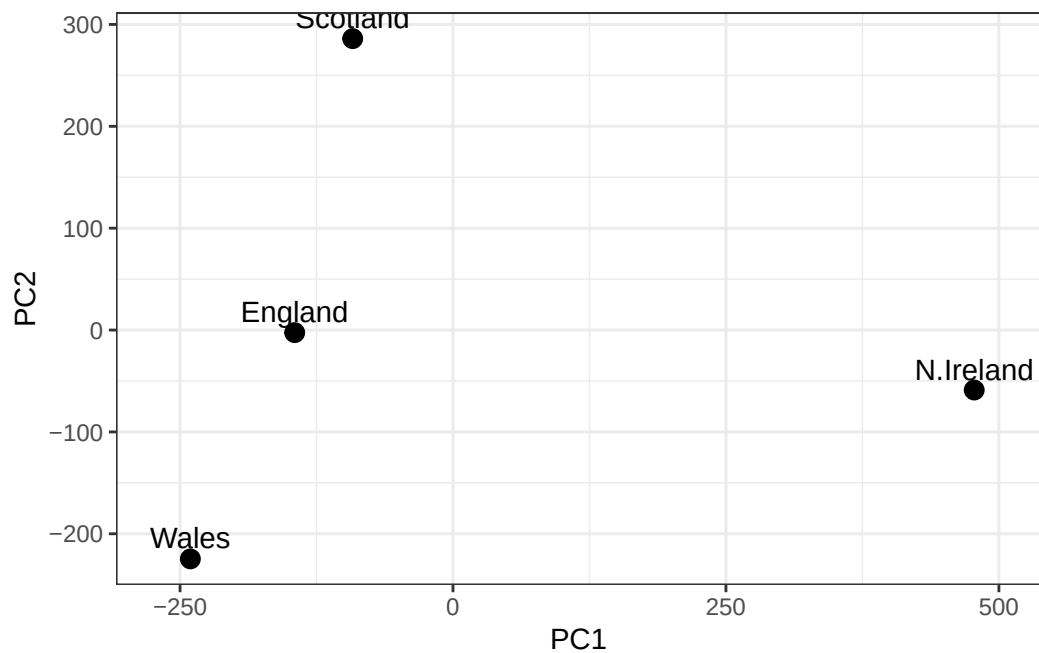
```
plot(pca$x[,1], pca$x[,2])
text(pca$x[,1], pca$x[,2], colnames(x))
```



Q7. Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

```
library(ggplot2)
# Create a data frame for plotting
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

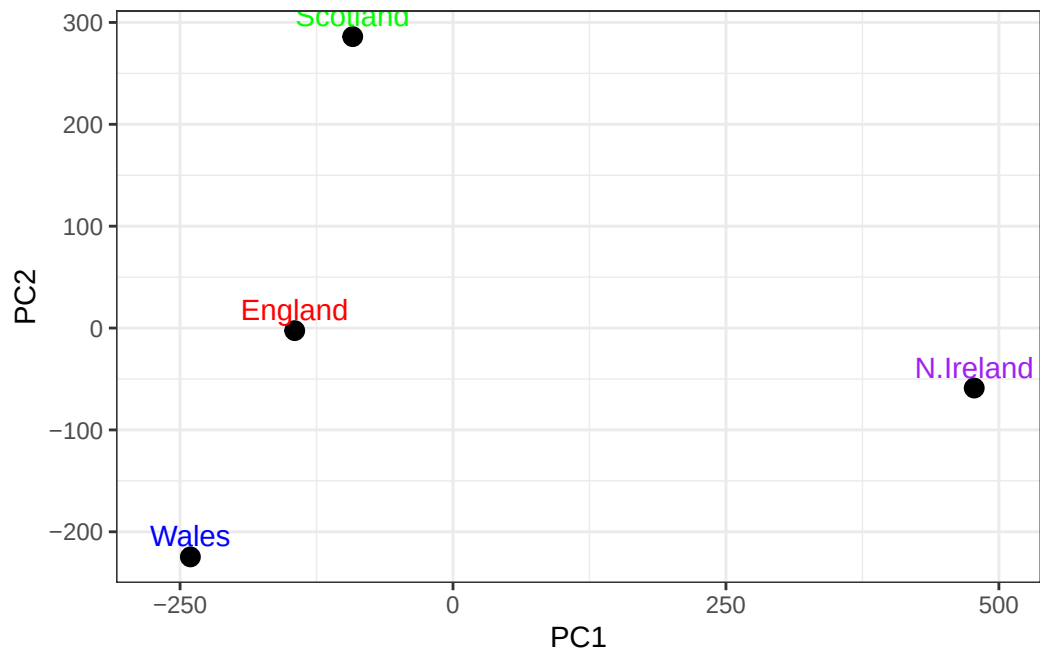
# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



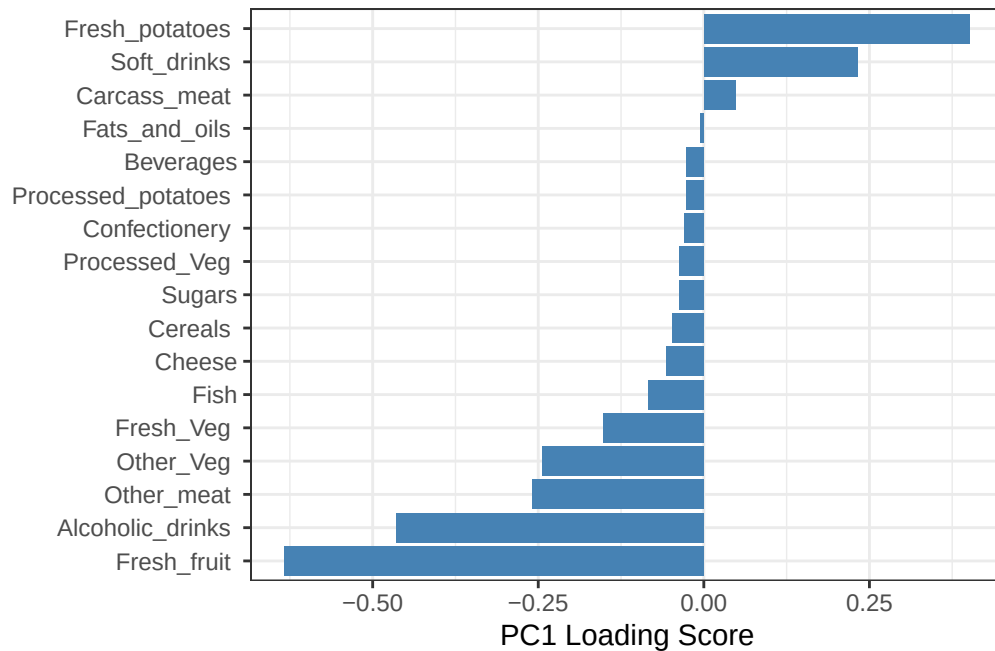
Q8. Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.

```
# Create a data frame for plotting
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(aes(label = df$Country),
    vjust = -0.5,
    color = c("red", "blue", "green", "purple")) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



```
## Lets focus on PC1 as it accounts for > 90% of variance
ggplot(pca$rotation) +
  aes(x = PC1,
      y = reorder(rownames(pca$rotation), PC1)) +
  geom_col(fill = "steelblue") +
  xlab("PC1 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



Q9. Generate a similar 'loadings plot' for PC2. What two food groups feature prominently and what does PC2 mainly tell us about?

Soft drinks and fresh potatoes feature most prominently. PC2 likely represents variation in countries with higher consumption of soft drinks and potatoes compared to countries with lower consumption of those foods.

```
# Generate a similar loadings plot for PC2
ggplot(pca$rotation) +
  aes(x = PC2,
       y = reorder(rownames(pca$rotation), PC2)) +
  geom_col(fill = "steelblue") +
  xlab("PC2 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```

